

Technical Companion and Experiment Record

Post-Training Qwen2.5-0.5B with SFT, Reward Modeling, Guarded PPO, and Archived Extensions

Dheeraj Dhillon

July 2026

Abstract

This companion preserves the engineering, experimental, and artifact-level record behind the shorter two-column project report. It is intentionally more operational: it records the custom-to-TRL migration, sequence-length studies, reward-model audits, PPO failure modes, checkpoint and Colab contracts, GPU memory redesign, full-suite evaluation, qualitative curation, and the implemented-but-unexecuted DPO extension. The concise project report remains the primary academic account; this document is the durable experiment archive.

The final pipeline trains QWEN2.5-0.5B-INSTRUCT on NVIDIA’s heterogeneous HELPSTEER3-PREFERENCE. A 4,096-token SFT and reward-model budget retains almost all long-form examples. One epoch of response-only LoRA SFT reaches 72.02% validation mean token accuracy, and an SFT-initialized two-epoch Bradley–Terry reward model reaches 65.62% pairwise accuracy on 1,917 held-out pairs. The selected policy is then trained for 12,032 on-policy rollouts and 188 PPO updates with 768-token responses, domain-balanced sampling, quantile-calibrated reward bounds, EOS-aware terminal scoring, smooth repeated-4-gram shaping, KL control, and an exact-resume contract.

The completed 2,017-prompt suite is deliberately mixed. The learned reward model prefers guarded PPO over Base in 817 comparisons (40.51%) and over SFT in 974 (48.29%). PPO nevertheless completes without empty-output or numerical collapse, has a 16.31% cap-hit rate, and crosses a 25% repeated-4-gram threshold in 16.16% of responses. An earlier 100-update TRL checkpoint scored better under the same learned proxy but was substantially longer, more capped, and more repetitive; it is retained here as evidence of reward-model exploitation rather than presented as the final result. The main contribution is therefore an inspectable RLHF system and an honest account of the tradeoff between proxy optimization, behavioral control, and qualitative response quality.

Contents

1	Introduction	4
1.1	Contributions	4
2	RLHF Formulation	5
2.1	Notation	5
2.2	Supervised fine-tuning	5
2.3	Pairwise reward modeling	5
2.4	KL-regularized PPO	5
3	Model, Data, and Preprocessing	6
3.1	Base model and preference data	6
3.2	Chat formatting and truncation	7

4	Implementation and Experimental Infrastructure	7
4.1	From a custom trainer to TRL	7
4.2	Direct preference optimization extension	8
4.3	Implementation details adopted from N+	9
4.4	Reproducibility and Colab execution	9
5	Final Training Configuration and Results	10
5.1	Supervised fine-tuning	10
5.2	Reward-model training	10
5.3	PPO alignment	11
5.4	Guarded 188-update follow-up and H100 memory redesign	12
6	Experimental Progression	13
7	Final Policy-Suite Evaluation	14
7.1	Protocol	14
7.2	Overall results	14
7.3	Domain behavior	15
8	Qualitative Audit and Response Explorer	15
8.1	Stopping and repetition	15
8.2	Deterministic triage	16
8.3	Curated 100 and complete public artifact	17
9	Discussion	17
9.1	What the final run demonstrates	17
9.2	Limitations	18
10	Future Work	18
10.1	Establish a controlled evaluation protocol	18
10.2	Improve reward-model reliability	19
10.3	Strengthen supervised fine-tuning	19
10.4	Redesign PPO around observed failures	20
10.5	Compare alternative preference objectives	20
10.6	Improve decoding and stopping	20
10.7	Add factuality, execution, and safety evaluators	20
10.8	Scale model and systems capacity deliberately	21
10.9	Reproducibility and artifact governance	21
10.10	Practical next sequence	21

11 Conclusion	21
A Detailed Historical Experiment Ledger	22
A.1 Initial validation and sequence-length diagnosis	22
A.2 Frozen custom SFT, reward, and PPO baseline	22
A.3 Historical policy-suite results	23
A.4 Historical A100 40GB profile	23
B Experiment Tracking and Run Governance	24
C Operational TRL, Colab, and Evaluation Contract	24
D Curation Workflow and Publication Standard	25
E Reproducibility and Artifact Map	25
F Metric Interpretation	27

1 Introduction

Large language models are trained primarily by next-token prediction, while users ultimately care about properties such as helpfulness, relevance, correctness, clarity, and safety. RLHF addresses this mismatch by learning a reward signal from preference comparisons and then optimizing a language model against that signal [1, 4, 5]. In its canonical form, the procedure has three stages: supervised fine-tuning on preferred demonstrations, training a reward model on chosen/rejected pairs, and policy optimization with a KL constraint to a reference policy.

This project deliberately pairs a modest compute budget with a nontrivial preference distribution. It asks whether the complete procedure can be made measurable and inspectable on a half-billion-parameter instruction model while retaining HelpSteer3’s general, code, STEM, multilingual, and long-form examples. The small policy makes iteration possible on rented notebook hardware, but it also makes the experiment a stress test of data quality, reward reliability, implementation detail, and evaluation discipline. The base model, QWEN2.5-0.5B-INSTRUCT, is already instruction-tuned and belongs to the Qwen2.5 family [8]. The work is therefore preference post-training rather than instruction tuning from an unaligned base.

The project also connects two settings that are often discussed separately. Earlier work in the same codebase implemented Trust Region Policy Optimization (TRPO), Natural Policy Gradient, and PPO for MuJoCo and Atari. TRPO constrains policy updates through a trust region [2]; PPO approximates the same conservative-update principle with a clipped surrogate objective [3]. In language-model RLHF, a second trust-region-like constraint appears through the KL penalty to a frozen SFT reference. The optimization machinery changes scale, but the central concern remains the same: improve expected reward without allowing destructive policy drift.

1.1 Contributions

The project makes six practical contributions:

1. It implements the full SFT–reward-model–PPO pipeline first with custom training code and then with TRL, preserving the custom lineage as a reproducible record of the assumptions and failures that motivated the final design.
2. It develops response-safe, long-context preprocessing for HELPSTEER3-PREFERENCE, including assistant-only loss masks, complete EOS-terminated completions, distinct PAD/EOS tokens, prompt-first truncation, and prompt-identity tracking across training and evaluation.
3. It initializes reward and value learning from the supervised policy and adopts high-impact PPO details such as zero dropout, matched behavior probabilities, fixed-length rollout masks, explicit missing-EOS treatment, and separate policy/reference semantics.
4. It adds guarded long-form PPO: balanced domain sampling, quantile-calibrated reward clipping, lower-bound scoring for missing EOS, smooth repetition shaping, KL regularization, health-gated checkpoints, and exact deterministic resume.
5. It redesigns the long-response execution path to avoid dense vocabulary-score retention, use rollout-only KV caching, recompute sampled token log-probabilities in bounded chunks, trim all-masked prompt columns, share the reference backbone, and split policy/value backward graphs.
6. It evaluates Base, SFT, and PPO once on the same 2,017 prompts and publishes complete responses, reward scores, stopping and repetition metadata, resumable policy shards, review queues, and a public response explorer rather than reporting aggregate reward alone.

The source repository is available at <https://github.com/djdhillxn/rlhf>. The response explorer is available at <https://djdhillxn.github.io/projects/rlhf-comparison/>.

2 RLHF Formulation

2.1 Notation

Let x denote a formatted conversational prompt and $y = (y_1, \dots, y_T)$ an assistant response. The trainable policy is $\pi_\theta(y \mid x)$, the frozen SFT reference is $\pi_{\text{ref}}(y \mid x)$, and the scalar reward model is $r_\phi(x, y)$. A response mask $m_t \in \{0, 1\}$ identifies generated assistant tokens so prompt and padding positions do not contribute to policy or value losses.

2.2 Supervised fine-tuning

SFT maximizes the likelihood of the preferred assistant response while masking the prompt:

$$\mathcal{L}_{\text{SFT}}(\theta) = - \sum_{t=1}^T m_t \log \pi_\theta(y_t \mid x, y_{<t}). \quad (1)$$

This is ordinary causal language modeling applied only to the completion. It serves two roles: it creates a policy adapted to the preference dataset, and it defines the reference distribution against which PPO drift is measured. Low-rank adaptation (LoRA) [9] updates small trainable matrices inside the attention and feed-forward projections while leaving the main backbone fixed.

2.3 Pairwise reward modeling

Each preference record contains a chosen response y^+ and a rejected response y^- for the same prompt. The reward model is trained with the Bradley–Terry logistic loss:

$$\mathcal{L}_{\text{RM}}(\phi) = - \log \sigma \left(r_\phi(x, y^+) - r_\phi(x, y^-) \right), \quad (2)$$

where σ is the sigmoid function. Pairwise accuracy is

$$\text{Acc}_{\text{RM}} = \frac{1}{N} \sum_{i=1}^N \mathbf{1} \left[r_\phi(x_i, y_i^+) > r_\phi(x_i, y_i^-) \right]. \quad (3)$$

The sign of an isolated score is not intrinsically meaningful: adding a constant to every reward leaves Equation 2 unchanged. This project therefore interprets margins and rankings rather than treating zero as a quality boundary. After training, a fixed demonstration-score offset is estimated and subtracted consistently during PPO and evaluation.

2.4 KL-regularized PPO

For a sampled response, the reward model supplies a terminal score while the reference policy supplies a token-level KL-shaped penalty:

$$r_t^{\text{KL}} = -\beta [\log \pi_\theta(y_t \mid x, y_{<t}) - \log \pi_{\text{ref}}(y_t \mid x, y_{<t})], \quad (4)$$

$$R(x, y) = r_\phi(x, y) + \sum_{t=1}^T r_t^{\text{KL}}. \quad (5)$$

The coefficient β controls policy drift. Generalized advantage estimation (GAE) computes

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t), \quad (6)$$

$$\hat{A}_t = \delta_t + \gamma \lambda \hat{A}_{t+1}, \quad (7)$$

with $\gamma = 1$ and $\lambda = 0.95$ in the final run. Advantages are whitened over valid response tokens. For old-policy log probability $\log \pi_{\text{old}}$ and updated-policy log probability $\log \pi_{\theta}$, define

$$\rho_t(\theta) = \exp(\log \pi_{\theta}(y_t | s_t) - \log \pi_{\text{old}}(y_t | s_t)). \quad (8)$$

PPO minimizes the negative clipped surrogate:

$$\mathcal{L}_{\text{policy}} = -\mathbb{E}_t \left[\min \left(\rho_t \hat{A}_t, \text{clip}(\rho_t, 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right], \quad (9)$$

with $\epsilon = 0.2$. The critic uses a clipped squared-error objective, and the combined optimization loss is

$$\mathcal{L} = \mathcal{L}_{\text{policy}} + c_v \mathcal{L}_{\text{value}} - c_H \mathcal{H}(\pi_{\theta}). \quad (10)$$

The final configuration uses $c_v = 0.1$. These equations were implemented directly in the custom trainer and later delegated to the experimental TRL PPO trainer. The repository continues to own model preparation, sampling controls, EOS handling, reward centering, checkpoint metadata, and evaluation.

3 Model, Data, and Preprocessing

3.1 Base model and preference data

HELPSTEER3-PREFERENCE is a human-annotated preference dataset spanning general instruction following, code, STEM, and multilingual tasks [7]. Its breadth is useful for a general alignment study, but it also makes the task harder than short-form summarization: many preferred answers contain code, multi-step explanations, or long conversational context.

Table 1: Final data and model profile.

Item	Value
Base policy	QWEN2.5-0.5B-INSTRUCT
Approximate parameter count	0.5B
Raw training preference records	38,459
Raw validation preference records	2,017
Filtered SFT/RM training records	36,264
Filtered SFT/RM validation pairs	1,917
Unique PPO training prompts	24,018
Policy-suite evaluation prompts	2,017
Maximum SFT/RM sequence length	4,096 tokens
Maximum PPO/evaluation prompt length	3,072 tokens
PPO rollout response length	768 tokens
Evaluation generation cap	1,024 tokens

Invalid or tied preferences remove 2,195 training records and 100 validation records from SFT/RM training. PPO uses prompt-only records and removes 14,441 duplicate training prompts. The

policy-suite evaluation is separate from this deduplicated PPO prompt cache and covers all 2,017 validation prompts.

3.2 Chat formatting and truncation

All stages use the Qwen chat template and the same tokenizer. The preprocessor constructs three views of each retained preference record:

- **SFT:** prompt plus chosen response, with a completion mask that excludes prompt tokens from Equation 1;
- **Reward model:** chosen and rejected sequences sharing the same prompt prefix;
- **PPO:** deduplicated, left-padded prompt token IDs.

PAD and EOS use distinct token IDs. Every SFT and reward-model completion ends in EOS. When a sequence exceeds the budget, the preprocessor removes the oldest non-system conversational turns first and left-truncates prompt tokens only as a final fallback. It does not cut a response merely to preserve more prompt history.

The project initially used a 1,024-token total limit. A direct length diagnostic showed that this discarded too much preference signal:

Table 2: Historical truncation diagnostic that motivated the 4,096-token training budget.

Limit	Train SFT	Train RM	Val. SFT	Val. RM
1,024	38.47%	40.82%	36.78%	39.49%
2,048	15.48%	16.47%	13.51%	14.87%
3,072	5.28%	5.83%	4.69%	5.32%
4,096	0.83%	1.00%	0.68%	0.89%

The final preparation report is consistent with this diagnostic: 304 of 36,264 SFT training prompts and 363 of 36,264 reward-model training prompts required truncation. The median chosen/SFT sequence length is 768 tokens and the 95th percentile is roughly 3,080 tokens. Long-context handling was therefore not cosmetic; it preserved a substantial part of the dataset.

4 Implementation and Experimental Infrastructure

4.1 From a custom trainer to TRL

The first implementation built the algorithm from low-level components: response masking, shifted token log-probabilities, token-level KL rewards, GAE, normalized advantages, clipped policy and value losses, adaptive KL control, LoRA checkpointing, and evaluation. This path was useful precisely because it failed in visible ways. Short response caps produced truncation; weak KL control or noisy adaptive estimates allowed drift; incorrect checkpoint paths invalidated comparisons; and high reward sometimes coincided with gibberish, empty outputs, or repeated text.

The final system uses Hugging Face TRL for trainer loops and standard serialization [10]. The migration was not a deletion of the project. Responsibilities were divided as follows:

Table 3: Ownership after migration to TRL.

Repository-owned	TRL-owned
HelpSteer3 parsing and filtering	SFT, reward, and PPO trainer loops
Qwen chat formatting and truncation	Mixed-precision/distributed preparation
Completion masks and prompt deduplication	Gradient accumulation and optimizer stepping
Model initialization and reward centering	PPO ratio, clipped objectives, and GAE
Fixed-length generation/EOS patches	Trainer checkpoint serialization
Run manifests and resolved configurations	Core training-loop metrics
Policy-suite evaluation and qualitative audit	PEFT trainer integration

The custom implementation remains frozen at Git commit:

`6cbf214fcf1b91c7b756e303e533c2c86d2eba89`

This preserves the learning history and provides a baseline for inspecting how trainer mechanics changed.

The complete repository immediately before the later legacy-script pruning is frozen at Git commit:

`6233219d2ee34517bc4203e0bb986f7cb27bf5d1`

That snapshot retains every custom training entry point and historical helper. The streamlined repository keeps only the active TRL SFT, reward, PPO, and DPO path plus the shared evaluation, audit, tracking, and synchronization closure. This second revision boundary makes removed educational code recoverable without carrying it in the current working surface.

4.2 Direct preference optimization extension

The next controlled experiment uses Direct Preference Optimization (DPO) [12] from the same frozen SFT checkpoint. This comparison changes the alignment objective while holding the policy initialization, preference source, tokenizer, sequence budget, LoRA parameterization, and evaluation prompts fixed. It therefore tests whether direct offline preference learning is more robust than optimizing the learned scalar reward with online PPO at this model scale.

The DPO cache stores explicit **prompt**, **chosen**, and **rejected** fields. It applies the same response-safe 4,096-token preprocessing as SFT and reward modeling: complete EOS-terminated responses are preserved, old prompt turns are removed first, and only a final prompt-side token fallback is allowed. Tied pairs are excluded. The primary run uses one row per non-tied pair because HelpSteer3’s overall preference levels $\{-3, \dots, 3\}$ are ordinal labels rather than established linear utility distances. Preference magnitude, individual annotator scores, and directional agreement remain attached as metadata for stratified analysis. A separately named linear-replication run is available only as a sensitivity experiment.

The active DPO implementation uses the Hugging Face TRL `DPOTrainer`, a standard sigmoid loss with $\beta = 0.1$, zero dropout, LoRA rank 16, a 4,096-token prepared-data budget, one epoch, and effective batch size 32. Reference log probabilities are precomputed, and the reference policy is the same SFT model with the trainable adapter disabled. On Colab, training runs under `/content` while the SFT model, prepared DPO data, periodic complete Trainer checkpoints, and final artifacts are synchronized to Google Drive. Checkpoints are written every 200 optimizer updates and `resume_from_checkpoint=auto` restores optimizer, scheduler, RNG, and Trainer state after a runtime loss. The controlling notebook is `notebooks/rlhf_dpo_colab_pipeline.ipynb`; the canonical configuration is `configs/trl/qwen25_05b_helpsteer3_dpo.yaml`.

4.3 Implementation details adopted from N+

RLHF is unusually sensitive to details that can look incidental in pseudocode. The N+ reproduction [6], which itself reconstructs the summarization setup of Stiennon et al. [4], guided the final configuration:

Table 4: High-impact RLHF/PPO implementation details in the final path.

Detail	Implementation
Disable PPO dropout	PyTorch dropout probabilities and LoRA dropout are zero, preventing stochastic ratio noise between rollout and update.
Match behavior probabilities	Generation temperature is applied consistently when storing and recomputing log probabilities; checkpoint generation heuristics are neutralized.
Preserve completions and EOS	Prompt history is truncated before response tokens; SFT/RM completions end in EOS.
Initialize RM from SFT	The merged SFT model supplies the reward backbone.
Initialize reward head deliberately	The scalar head uses zero bias and standard deviation $1/\sqrt{d_{\text{hidden}} + 1}$.
Center rewards	Training uses a centering regularizer, then subtracts a persisted mean demonstration score.
Initialize critic from RM	The PPO value model begins from the trained reward-model weights.
Use a frozen SFT reference	PPO adapter weights are disabled for reference log probabilities.
Fixed-length EOS trick	Rollouts sample the configured length without stopping at EOS, are truncated at EOS afterward. The evaluated historical run used reward -1 when EOS was missing; the guarded follow-up instead replaces the score with the calibrated 0.5th-percentile lower reward boundary.
Whiten rewards/advantages	The guarded follow-up disables optional reward whitening so calibrated terminal penalties retain their scale; TRL continues to whiten masked advantages.
Adam numerical setting	Adam epsilon is 10^{-5} .

These choices reduce avoidable instability; they cannot repair a weak reward model, noisy preferences, or insufficient model capacity.

4.4 Reproducibility and Colab execution

Each stage writes a resolved YAML configuration, experiment manifest, summary, trainer metrics, and stage-specific artifacts. Smoke runs precede expensive runs. A “doctor” script validates the Python interpreter, CUDA visibility, trainer APIs, prepared datasets, tokenizer special tokens, and local/Drive write paths.

The Colab workflow keeps active datasets and checkpoints on local SSD for throughput and synchronizes periodic checkpoints and final artifacts to Google Drive. SFT and reward training support exact Transformers checkpoint resume. The guarded PPO wrapper adds an explicit exact-resume layer around the experimental TRL trainer. Every verified checkpoint contains the LoRA policy, complete value model, optimizer, scheduler, Trainer state, Python/NumPy/CPU/CUDA RNG state, deterministic rollout-plan position, calibrated guardrails, and a health marker written only after all required files exist. Training is divided into cumulative target updates and resumes from the latest verified checkpoint; an incomplete newer directory is ignored.

Evaluation is resumable at the policy-output level. Each policy writes complete JSONL records as generation proceeds; “partial” denotes interrupt-safe stage output, not truncated response text. A fingerprint guards against reusing cached responses under a changed checkpoint or generation configuration.

5 Final Training Configuration and Results

5.1 Supervised fine-tuning

SFT trains LoRA modules in the attention projections (**q**, **k**, **v**, and **o**) and feed-forward projections (**gate**, **up**, and **down**). Prompt and padding tokens are masked from the loss, so the objective applies only to the preferred assistant response. The effective batch size is $8 \times 4 = 32$ on one device.

Table 5: Final SFT configuration and outcome.

Setting or metric	Value
Trainer	TRL SFTTrainer
Epochs	1
LoRA rank / alpha	16 / 32
LoRA dropout	0
Effective batch size	32
Learning rate	5×10^{-6}
Scheduler	cosine, 3% warmup
Precision	bfloat16, TF32 enabled
Maximum total length	4,096 tokens
Train / validation loss	1.0556 / 1.1127
Validation mean token accuracy	72.02%

The merged SFT checkpoint is used in three roles: as a comparison policy, as the initialization for the reward model and PPO policy, and as the frozen PPO reference. That common starting point makes downstream changes easier to attribute.

5.2 Reward-model training

The reward model starts from the merged SFT backbone and trains LoRA rank-32 adapters plus a scalar head using the Bradley–Terry loss. A persisted scalar offset centers preferred demonstrations consistently in PPO and evaluation; it does not change pairwise ordering.

Table 6: Final two-epoch reward-model audit.

Metric	Value
Usable validation pairs	1,917
Validation loss	0.6166
Pairwise accuracy	65.62%
Mean chosen–rejected margin	0.3578
Persisted score offset	−0.1985
Code / General accuracy	70.23% / 62.91%
STEM / Multilingual accuracy	59.26% / 71.82%

The domain spread matters for interpretation. The model is useful as a broad preference proxy, but 65.62% accuracy—and especially the weaker STEM and general subsets—is not sufficient to treat its scalar score as ground truth. Only score differences are identifiable; negative absolute scores do not by themselves indicate a bad response.

5.3 Final guarded PPO run

The selected PPO run was designed around long-form responses rather than around maximizing the reward model at any cost. It completed the configured 12,000- episode budget as 12,032 rollouts: 188 batches of 64 responses. Each batch contains 16 code, 16 general, 16 STEM, and 16 multilingual prompts. Approximately 12,007 prompt rows are unique, so the run covers a broad slice of the dataset without pretending to be a conventional supervised epoch.

The terminal score is clipped to empirical 0.5th and 99.5th percentiles estimated from 4,096 stratified reward-training pairs, $[-2.7507, 1.5018]$. A response that fails to produce EOS receives the lower bound, which makes nontermination as unattractive as the worst calibrated response rather than assigning an arbitrary weak penalty. A smooth repeated word-level 4-gram penalty activates only above 0.3084, the preferred-response 95th percentile. This protects ordinary repeated terminology while shaping clear loops. The PPO objective retains a frozen SFT reference, KL coefficient 0.10, clipped policy and value losses, an RM-initialized critic, advantage normalization, and four PPO epochs per rollout batch. Scalar reward whitening is disabled so the calibrated reward scale remains meaningful.

Table 7: Final guarded PPO configuration and terminal diagnostics.

Setting or metric	Value
Prompt / response limits	3,072 / 768 tokens
Completed rollouts / updates	12,032 / 188
Rollout batch / PPO epochs	64 / 4
Domain composition per update	16 / 16 / 16 / 16
Learning rate	3×10^{-6} , linear decay
KL coefficient	0.10
Calibrated reward interval	$[-2.7507, 1.5018]$
Repetition activation threshold	0.3084
Final objective KL to SFT reference	0.7305
Final objective KL per response token	0.00199
Final old-to-new approximate KL	0.000537
Final clip fraction	0.00571
Final value loss	0.1076
Final rollout mean / median tokens	366.84 / 342.5
Final rollout EOS / cap rate	82.81% / 17.19%
Final rollout repeated-4-gram fraction	17.0%
Empty-response rate	0%
Peak allocated CUDA memory	37.64 GiB

Across updates 1–94 and 95–188, mean reward-model score improves from -1.110 to -0.961 , EOS rate rises from 74.3% to 81.0%, cap rate falls from 25.7% to 19.0%, mean response length falls from 452 to 409 tokens, and mean repeated-4-gram fraction falls from 22.2% to 19.5%. Mean value loss falls from 2.83 to 0.114 as the critic adapts to the reward scale. These trends support a narrow statement: the run learned and remained numerically healthy. They do not establish that the learned reward is an independent measure of human preference.

5.4 Memory-efficient long-response execution

Long responses made systems design part of the PPO method. Stock experimental TRL retained a dense generation-score tensor with shape $64 \times 768 \times 151,665$, approximately 27.77 GiB in float32 before the policy update. The final execution path instead:

- generates token IDs without retaining per-step vocabulary scores;
- uses a rollout-only KV cache, then recomputes behavior log-probabilities only for sampled response tokens;
- bounds the recomputation with `logits_to_keep=769` and response-position chunking;
- dynamically drops prompt columns masked for every item in the batch;
- shares the reference backbone by evaluating with the policy adapter disabled rather than loading a second full backbone; and
- backpropagates policy and value losses through separate graphs before one optimizer step, reducing peak activation retention.

These changes preserve the rollout distribution, reward, generalized advantages, clipping, domain balance, and effective batch size. They change only how equivalent quantities are materialized. Together with bfloat16 and TF32, they make 768-token, batch-64 PPO practical with 37.64 GiB peak allocated memory and no generation or log-probability OOM fallback in the completed run.

6 Experimental Progression

The project did not move directly from a first implementation to the final run. Table 8 records the main stages; detailed custom metrics are retained in Appendix A.

Table 8: Selected experimental progression. Only the last row is the final reported PPO result.

Stage	Budget	What it established
Short custom smoke tests	96–128 output tokens	Validated masks, KL, value learning, and checkpointing, while exposing truncated examples, gibberish, empty responses, and invalid checkpoint loading.
Custom long-512 PPO	397 updates, batch 2	Produced complete responses but weak aggregate preference results; motivated full-suite evaluation and stronger stopping diagnostics.
Custom 1,024-token evaluation	2,017 prompts	Reduced truncation but exposed high-reward loops, fabricated details, and reward-model mismatches; not a clean cap ablation.
TRL checkpoint 100	6,400 rollouts, 100 updates	Reached 50.92% proxy win rate versus Base and 57.71% versus SFT, but with 520-token median responses, 27.42% cap hits, and 31.88% above the 25% repetition threshold. It demonstrated stronger proxy optimization and clearer reward exploitation, not a final aligned policy.
Final guarded TRL PPO	12,032 rollouts, 188 updates	Completed the declared budget with calibrated rewards, EOS and repetition guards, balanced domains, exact resume, and the memory-efficient long-response path. This is the final reported policy.

The checkpoint-100 experiment remains important. It is the clearest evidence that the learned reward can be improved while visible behavior deteriorates. Relative to that archived run, the final guarded evaluation is shorter and less capped and repetitive, but it also loses the earlier reward-model lead. Because generation caps and other execution details differ, the comparison is an engineering progression rather than a controlled causal ablation.

7 Final Policy-Suite Evaluation

7.1 Protocol

Base, SFT, and guarded PPO each generate one response for every one of the 2,017 validation prompts. Generation uses the same tokenizer, prompt formatting, 3,072-token prompt limit, 768-

token response limit, temperature 0.7, and top-p 1.0. The two-epoch reward model scores every response. Evaluation writes one JSONL shard per policy as it proceeds and can resume without regenerating completed rows. All pairwise comparisons are derived from the same shared response table, eliminating inconsistent prompt samples across comparisons.

The reward-model winner is useful but incomplete. The report therefore places length, cap, EOS, and repeated-4-gram diagnostics before the win statistics and retains full responses for human inspection.

7.2 Overall results

Table 9: Behavior-first full-suite summary on 2,017 prompts.

Policy	Behavior					Learned reward		
	Mean tok.	Med. tok.	Cap	Rep. > 25	Rep. > 50	Mean	Wins	Rate
Base	373.5	333	16.21%	165 (8.18%)	43 (2.13%)	0.2229	845	41.89%
SFT	392.5	364	18.34%	374 (18.54%)	143 (7.09%)	0.2037	585	29.00%
PPO	384.1	355	16.31%	326 (16.16%)	105 (5.21%)	0.1753	563	27.91%
Tie	—	—	—	—	—	—	24	1.19%

Table 10: Pairwise reward-model comparisons.

Comparison	Left	Right	Ties	Right rate	Mean Δ
Base vs SFT	1,145	849	23	42.09%	−0.0192
Base vs PPO	1,179	817	21	40.51%	−0.0476
SFT vs PPO	1,018	974	25	48.29%	−0.0284

PPO is closest to SFT, but the reward proxy does not prefer it in aggregate. The mean deltas are small and the win counts still favor Base and SFT. This is not a hidden victory and should not be advertised as one. It is evidence that the policy remained competitive while the guardrails constrained several behaviors that the earlier checkpoint had exploited.

7.3 Domain behavior

Table 11: Guarded PPO versus Base by prompt domain.

Domain	Prompts	PPO wins	Base wins	Ties	PPO rate
Code	438	157	279	2	35.84%
General	931	400	517	14	42.96%
STEM	245	96	148	1	39.18%
Multilingual	403	164	235	4	40.69%

PPO does not beat Base within any domain. Against SFT, however, it wins the STEM subset 133 to 109 with three ties. This reinforces the need to inspect both the comparison policy and the task domain rather than collapse the whole project into one scalar win rate.

8 Qualitative Audit and Response Explorer

For response word-level 4-gram multiset $\mathcal{G}$, the repeated fraction is

$$f_{\text{rep4}} = 1 - \frac{|\text{unique}(\mathcal{G})|}{|\mathcal{G}|}. \quad (11)$$

It is a review heuristic, not a semantic metric: legitimate code and technical prose repeat terms, while some loops change punctuation enough to evade it.

The final audit creates overlapping navigation queues rather than mutually exclusive quality labels: 9 qualified PPO candidates, 120 strong PPO-loss candidates, 326 repetition-risk rows, and 155 high-reward repetition or stopping mismatches. The public explorer exposes all 2,017 prompts, complete Base and PPO responses, reward scores and margins, token counts, EOS/cap state, repetition diagnostics, domain filters, and a balanced curated first pass. Reward scores are explicitly labeled as diagnostics, not human judgments.

Manual inspection finds useful local changes: clearer structure, better coverage of some multi-part instructions, supportive framing, and more compact answers in selected cases. It also finds failures that the learned reward does not reliably detect: invalid code, confused factual or scientific reasoning, prompt restatement, fabricated details or citations, irrelevant continuation, and occasional near-cap loops that still receive high reward. These examples are the practical evidence for reward-model overoptimization discussed more generally by Gao et al. [11].

The explorer is therefore not a promotional gallery of PPO wins. It is the bridge from aggregate statistics to the actual object being optimized: model text. The repository retains both positive and negative queues so a reader can inspect where the proxy agrees with visible quality and where it does not.

9 Discussion

The completed run supports a strong systems conclusion and a modest alignment conclusion. The system joins long-context preprocessing, response-only SFT, pairwise reward learning, online PPO, calibrated safeguards, exact resume, memory-bounded long-response execution, full-suite generation, and qualitative audit on a heterogeneous preference dataset. The optimizer behaves as intended: losses remain finite, old-to-new approximate KL and clip fraction are small, EOS behavior improves during training, empty outputs remain absent, and the critic converges to the reward scale.

That stability is not equivalent to model improvement. The same imperfect reward model supplies the training reward and the final quantitative judge; there is no blinded multi-rater human study, no independent frontier-model judge, and only one sampled generation per policy and prompt. Response length remains a confound, the 0.5B policy and reward model have limited capacity, and word-level repetition heuristics cannot establish factuality, safety, or instruction following. Historical token-budget and checkpoint comparisons also change more than one variable and should be read as operational evidence, not clean ablations.

The result is consequently framed without hiding the negative evidence. PPO changed behavior and produced useful examples, but guarded PPO did not beat Base or SFT under the learned proxy. The project succeeds because it makes that outcome diagnosable and reproducible, and because the earlier higher-proxy checkpoint is preserved as an explicit example of why reward optimization alone is not enough.

10 Future Work

The highest-value next step is independent evaluation rather than another unqualified PPO continuation. A blinded human or strong-LLM comparison on a stratified sample should measure agreement with the learned reward model and identify hard negatives for reward-model retraining. Code responses should be executed where possible; STEM and factual claims should receive task-specific checks. A controlled decoding study can then compare 512, 768, and 1,024-token budgets while holding the checkpoint, prompt order, seed, batch size, and software environment fixed.

Only after the judge is stronger should PPO be extended or restarted. Useful variants include a response-length curriculum, checkpoint selection based on a joint scorecard of reward, KL, EOS, cap, repetition, and independent preference, and larger reward/policy models. The prepared DPO implementation provides an offline preference baseline from the same SFT checkpoint and data, but it is an archived extension until it is actually trained and evaluated under the same suite.

11 Conclusion

This project completes and audits an end-to-end RLHF pipeline for a small instruction model on heterogeneous, long-form preference data. Long-context SFT retains nearly all responses; the reward model learns a usable but imperfect pairwise signal; guarded PPO completes 12,032 rollouts with stable optimization, health-gated resume, explicit anti-exploitation controls, and a practical long-response memory path. The full evaluation does not show a PPO win over Base. It shows something more useful for this project: a controlled policy that can be inspected alongside the proxy failures and qualitative tradeoffs that shaped it.

The durable result is therefore not a single reward number. It is the combined training, systems, evaluation, and audit record—including the runs that looked better to the reward model but worse on stopping and repetition. That record is a defensible conclusion for this phase and a concrete foundation for future alignment experiments.

A Detailed Historical Experiment Ledger

This appendix retains the operational facts behind the compact progression in Table 8. Historical paths and configurations are evidence of what was run; they are not active recommendations.

A.1 Initial validation and sequence-length diagnosis

Early 96- and 128-token generations accelerated debugging but often stopped inside code blocks or explanations. These runs exposed multilingual or gibberish drift, vulgar debug-pattern text, EOS and blank-response collapse, overaggressive reward shaping, and invalid comparisons caused by incorrect checkpoint paths. The resulting safeguards included prompt sanitation, KL anchoring, empty-response checks, repeated-text diagnostics, and explicit checkpoint validation.

The sequence-length diagnostic established why the project moved from 1,024 to 4,096 total tokens:

Table 12: Fraction of records requiring truncation at each total-token limit.

Limit	Train SFT	Train RM	Valid. SFT	Valid. RM
1,024	38.47%	40.82%	36.78%	39.49%
2,048	15.48%	16.47%	13.51%	14.87%
3,072	5.28%	5.83%	4.69%	5.32%
4,096	0.83%	1.00%	0.68%	0.89%

A.2 Frozen custom SFT, reward, and PPO baseline

The pre-TRL lineage is frozen at commit `6cbf214fcf1b91c7b756e303e533c2c86d2eba89`. Custom SFT used QWEN2.5-0.5B-INSTRUCT, 4,096 total tokens, two epochs, batch size 6, accumulation 3, learning rate 5×10^{-6} , and LoRA rank 16. It completed 4,030 optimizer steps and wrote to `outputs/rlhf/qwen25_05b_helpsteer3_sft_4096/`.

The two-epoch custom reward model wrote to `outputs/rlhf/qwen25_05b_helpsteer3_reward_4096_epoch2/` and produced the following held-out audit:

Table 13: Historical custom reward-model result.

Metric	Value
Validation pairs	1,917
Accuracy / loss	71.62% / 0.9734
Mean chosen-rejected margin	0.9094
Code / General accuracy	74.88% / 71.01%
STEM / Multilingual accuracy	63.37% / 75.15%

The custom long-512 PPO run used a 3,072-token prompt budget, 512-token rollouts, learning rate 3×10^{-7} , policy clip 0.06, one PPO epoch, rollout batch 2, and an adaptive KL coefficient initialized at 0.18 with a 0.14 minimum. It completed 397 of 400 requested updates and ended at KL coefficient 0.14. Rewards were clipped to $[-5, 5]$; scalar reward whitening was absent; length penalty was 0.002; missing-EOS penalty was zero; responses shorter than 32 tokens received a 0.5 penalty; group-relative normalization was disabled. Generation applied repetition penalty 1.05 and blocked repeated 4-grams. Prompt batches were left padded, and response masks excluded padding from PPO losses. Its output is `outputs/rlhf/qwen25_05b_helpsteer3_ppo_4096_epoch2_long512/`.

A.3 Historical policy-suite results

The 512-token suite used all 2,017 validation prompts. Three-way wins were Base 827 (41.00%), SFT 556 (27.57%), PPO 525 (26.03%), and 109 ties (5.40%). Mean rewards were -3.5339 , -3.4280 , and -3.6666 ; median response lengths were 332, 363, and 356 tokens; and cap-hit rates were 29.90%, 29.95%, and 29.70%. Its complete pairwise table is retained below.

Table 14: Historical custom 512-token pairwise evaluation.

Comparison	Left	Right	Ties	Right rate	Mean Δ
Base vs SFT	1,044	927	46	45.96%	+0.1059
Base vs PPO	1,068	904	45	44.82%	-0.1327
SFT vs PPO	965	898	154	44.52%	-0.2386

The later 1,024-token suite produced three-way wins of Base 978 (48.49%), SFT 475 (23.55%), PPO 467 (23.15%), and 97 ties (4.81%). Mean rewards were -3.3634 , -3.6114 , and -3.5771 ; median lengths were 334, 360, and 363; cap-hit rates were 8.08%, 10.16%, and 11.60%; empty rates were zero.

Table 15: Historical custom 1,024-token pairwise evaluation.

Comparison	Left	Right	Ties	Right rate	Mean Δ
Base vs SFT	1,215	763	39	37.83%	-0.2480
Base vs PPO	1,190	785	42	38.92%	-0.2137
SFT vs PPO	963	892	162	44.22%	$+0.0343$

PPO’s 785 Base wins had mean margin $+1.6210$, while its 1,190 losses had mean margin -1.4315 . Against SFT, its 892 wins averaged $+1.3503$ and its 963 losses -1.1789 . Domain PPO win rates against Base were 26.26% for code (115/438), 44.47% for general (414 wins, 487 losses, 30 ties), 39.59% for STEM (97/245 with two ties), and 39.45% for multilingual (159 wins, 236 losses, eight ties).

Increasing the evaluation cap reduced Base/SFT/PPO cap hits from 603/604/599 to 163/205/234. It also exposed more PPO repetition: 224 responses exceeded 25% repeated 4-grams in the 512-token suite versus 325 (16.11%) in the 1,024-token suite; severe cases above 50% rose from 74 to 156. Manual review found fabricated citations, incorrect chemistry, irrelevant continuations, and high-reward loops. This was not a clean token-cap ablation: batch size changed from 8 to 128, and bfloat16 batching, environment, or evaluator revisions could change close greedy continuations.

A.4 Historical A100 40GB profile

An earlier speed-oriented Colab profile used full bfloat16 weights rather than 4-bit loading. Reward training used batch 8, accumulation 2 (effective 16), all filtered HelpSteer3 pairs, and refreshed token throughput, CUDA memory, CSV files, and plots every configured artifact interval. Batch 12 was the suggested increase below roughly 28–30GB peak memory; batch 6 was the fallback near 36GB or after OOM.

Historical PPO used full bfloat16 policy/reference/reward models, rollout batch 16, minibatch 4, prompt length 768, response length 160, checkpoints every 50 updates, and metric refresh every 25 updates. Rollout batch 24 was the first recommended utilization increase below 30–32GB; minibatch 6 or 8 was reserved for runs with measured headroom. The early evaluator covered 200 prompts; the completed suite superseded it with all 2,017.

B Experiment Tracking and Run Governance

The project uses a filesystem contract rather than a tracking server. Every new run directory contains the following where applicable:

Table 16: Run-tracking contract.

Artifact	Purpose
<code>config_resolved.yaml</code>	Exact configuration consumed by the run.
<code>experiment_manifest.json</code>	Identity, intent, config hash, command, source state, environment, attempts, status, and summary.
<code>run_metadata.json</code>	Backward-compatible hardware/software record.
<code>*_metrics.jsonl</code> / CSV	Step-level measurements.
Run or evaluation summary	Final aggregate result.
Checkpoints, samples, plots	Stage-specific evidence.

An interrupted process leaves its manifest in **running** state. Reusing the directory appends an attempt and records whether the resolved configuration changed; successful completion updates status and summary. Configs can declare non-behavioral intent—experiment ID, name, group, tags, hypothesis, and parent runs—while algorithmic choices remain in functional sections such as generation, reward shaping, KL, and PPO.

Runs are compared with `python -m scripts.rlhf_compare_runs RUN_A RUN_B`; metadata-only fields are hidden by default. The command supports `-include-metadata`, `-format json`, and `-output comparison.md`. The frozen custom baseline remains under `experiments/baselines/qwen25_05b_helpsteer3_ppo_long512/`; its YAML record is authoritative after removal of its redundant prose README.

The final records live under `rlhf_runs/qwen25_05b_helpsteer3_trl_a100_full/full/`. Stage sources of truth are the SFT `run_summary.json`, reward epoch-1 and epoch-2 summaries and audits, the guarded PPO `run_summary.json`, the health-verified `checkpoint-188`, and the final 2,017-row policy-suite summary, sample table, and audit queues. Historical checkpoint-100 artifacts remain available only for the experimental progression and are not the final policy. Persisted resolved configuration and the executed guarded PPO notebook take precedence over obsolete labels in older directories.

C Operational TRL, Colab, and Evaluation Contract

The stage datasets share one filtered source but have distinct contracts: SFT stores token IDs and a completion mask; reward modeling stores paired chosen and rejected token IDs with a common prefix; PPO stores deduplicated left-padded prompts; DPO stores explicit prompt/chosen/rejected text plus preference-strength and annotator metadata. Prompt history is shortened before any final prompt-side truncation, responses retain EOS, and PAD is distinct from EOS.

The preflight doctor validates the exact interpreter, package versions, CUDA, trainer APIs, tokenizer special tokens, prepared data, and local/Drive write permissions before expensive work. Evaluation always loads the SFT tokenizer; Base is resized once for its distinct PAD token. Neutral generation controls are explicit so checkpoint defaults cannot create asymmetric decoding. Policy list overrides accept either dotted or bracketed indices. The policy labeled Base must keep a null checkpoint and load the configured model name; pointing it at SFT would silently make the Base and SFT columns identical.

Active work uses Colab local SSD for model shards, datasets, logs, and Trainer state, with periodic checkpoints and final artifacts copied to mounted Drive. This is faster and keeps model tensors outside Git. SFT, reward, and DPO support exact Trainer resume. DPO’s `auto` mode restores model, optimizer, scheduler, RNG, and Trainer state. The guarded wrapper supplies the missing protocol around experimental PPO: policy/value state, optimizer, scheduler, Trainer state, RNG state, rollout position, and guardrail fingerprints must all pass a health check before a directory is

eligible for automatic continuation.

Evaluation is resumable per policy. Its “partial policy outputs” are complete JSONL rows written incrementally; *partial* describes interrupt-safe run completion, not shortened prompts or responses. A fingerprint prevents cached rows from being reused when checkpoint or generation settings change.

TRL PPO APIs are experimental, and the repository intentionally validates the installed stack instead of assuming that an unpinned upgrade is compatible. Every dependency change requires a smoke run and a recorded environment before an expensive experiment.

D Curation Workflow and Publication Standard

The final workflow treats curation as review navigation rather than a source of headline metrics. The following artifacts are generated directly from the 2,017-row shared policy table:

Table 17: Curation artifacts retained by the final workflow.

Artifact	Purpose
<code>policy_suite_samples.jsonl / CSV / XLSX</code>	Full prompts, Base/SFT/PPO responses, learned rewards, lengths, EOS/cap state, and pairwise deltas.
<code>qualitative_audit_auto.md</code>	Aggregate stopping/repetition audit and candidate summary.
<code>curation_qualified_ppo_candidates.csv</code>	Low-warning PPO candidates for positive review.
<code>curation_strong_ppo_losses.csv</code>	Large learned-reward regressions.
<code>curation_ppo_repetition_risks.csv</code>	Responses above the configured repeated-4-gram review threshold.
<code>curation_reward_model_mismatches.csv</code>	High learned reward combined with repetition or stopping warnings.
<code>portfolio_full_policy_comparisons_final_trl.json</code>	Redacted full response-explorer payload.
<code>portfolio_curated_100_manifest_final_trl.json</code>	Balanced public first-pass subset.

The queues overlap and are deterministic functions of recorded diagnostics; they are not manual, human, or LLM-as-a-judge labels. Review must cover apparent wins, near ties, strong losses, repetition, proxy mismatches, correctness-sensitive code and STEM cases, and multilingual drift. Before calling an example an improvement, a reviewer checks instruction following, correctness, completeness, relevance, repetition, safety, and whether the reward advantage reflects substance rather than length or formatting. Publication must show both gains and failures. The defensible conclusion is that guarded PPO is stable and inspectable but does not beat Base under the learned judge.

E Reproducibility and Artifact Map

Table 18: Primary project artifacts used in this report.

Artifact	Purpose
Project overview	<code>README.md</code> . Concise final guarded-run story, training profile, full-suite results, interpretation, and links.
Repository guide	<code>docs/REPOSITORY_GUIDE.md</code> . Installation, commands, directory map, configuration conventions, checkpoint semantics, evaluation resume, and the archived DPO extension.

Artifact	Purpose
Academic report	<code>docs/rlhf_project_report.tex</code> and PDF. Concise scientific account of SFT, reward modeling, guarded PPO, memory execution, and the final evaluation.
Technical record	<code>docs/rlhf_technical_companion.tex</code> and PDF. Chronological experiments, trainer migration, earlier PPO results, operational contracts, DPO design, audit, and artifact governance.
Guarded PPO notebook	<code>notebooks/rlhf_ppo_guarded_colab_pipeline.ipynb</code> . Executed 188-update PPO workflow and synchronization steps.
Evaluation notebook	<code>notebooks/rlhf_full_eval_and_curation.ipynb</code> . Full-suite analysis and curation workflow.
Final SFT summary	<code>rlhf_runs/qwen25_05b_helpsteer3_trl_a100_full/full/sft/run_summary.json</code> .
Final reward audit	<code>rlhf_runs/qwen25_05b_helpsteer3_trl_a100_full/full/reward_epoch2/reward_audit.json</code> .
Final PPO summary	<code>rlhf_runs/qwen25_05b_helpsteer3_trl_a100_full/full/ppo_guarded_r768_b64_kl10_rm_ep2/run_summary.json</code> .
Final checkpoint	<code>.../ppo_guarded_r768_b64_kl10_rm_ep2/checkpoint-188/</code> . LoRA policy, value model, optimizer, scheduler, Trainer state, RNG state, rollout position, guardrail calibration, and health marker.
Final policy suite	<code>rlhf_runs/qwen25_05b_helpsteer3_trl_a100_full/full/eval_ppo_guarded_r768_b64_kl10_rm_ep2_u188_full2017/</code> . Authoritative 2,017-prompt summary, full rows, plots, and audit queues.
Full response artifact	<code>rlhf_runs/portfolio_full_policy_comparisons_final_trl.json</code> . Redacted Base/PPO explorer payload.

The active command sequence is:

```
python -m scripts.rlhf_trl_prepare_data \
  --config configs/trl/qwen25_05b_helpsteer3_sft.yaml
python -m scripts.rlhf_trl_train_sft \
  --config configs/trl/qwen25_05b_helpsteer3_sft.yaml
python -m scripts.rlhf_trl_train_reward_model \
  --config configs/trl/qwen25_05b_helpsteer3_reward.yaml
python -m scripts.rlhf_trl_train_ppo \
  --config configs/trl/qwen25_05b_helpsteer3_ppo.yaml
python -m scripts.rlhf_evaluate_policy_suite \
  --config configs/trl/qwen25_05b_helpsteer3_eval_suite.yaml
python -m scripts.rlhf_audit_policy_suite \
  --eval-dir \
  rlhf_runs/qwen25_05b_helpsteer3_trl_a100_full/full/\
  eval_ppo_guarded_r768_b64_kl10_rm_ep2_u188_full2017 \
  --base-label base --sft-label sft_trl --ppo-label ppo_trl
python -m scripts.rlhf_sync_runs --mode lightweight
```

The DPO preparation, doctor, training, and evaluation commands are retained in `docs/REPOSITORY_GUIDE.md` and the DPO subsection of this companion. They are not part of the executed final result. The executed guarded PPO notebook and persisted resolved configuration are authoritative when a historical path name conflicts with a runtime override.

F Metric Interpretation

SFT mean token accuracy

Fraction of preferred completion tokens predicted correctly under teacher forcing; not response-level

task accuracy.

Reward-model pairwise accuracy

Fraction of held-out pairs for which the chosen response scores above the rejected response.

Policy win rate

Fraction of evaluation prompts for which one policy receives the higher learned reward. It is a proxy-judge result, not a human preference rate.

Objective KL

Sampled response-level departure from the frozen SFT reference. It is not the same statistic as PPO's old-to-new update approximate KL.

Cap hit

The response uses the full evaluation generation budget and does not stop naturally before 768 new tokens.

Repeated 4-gram fraction

Fraction of word-level 4-gram positions attributable to repeated 4-grams, used only to prioritize qualitative review.

References

- [1] P. F. Christiano, J. Leike, T. Brown, M. Martic, S. Legg, and D. Amodei. Deep reinforcement learning from human preferences. In *Advances in Neural Information Processing Systems*, 2017. <https://arxiv.org/abs/1706.03741>.
- [2] J. Schulman, S. Levine, P. Abbeel, M. Jordan, and P. Moritz. Trust Region Policy Optimization. In *Proceedings of the 32nd International Conference on Machine Learning*, 2015. <https://arxiv.org/abs/1502.05477>.
- [3] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal Policy Optimization Algorithms. arXiv:1707.06347, 2017. <https://arxiv.org/abs/1707.06347>.
- [4] N. Stiennon, L. Ouyang, J. Wu, D. Ziegler, R. Lowe, C. Voss, A. Radford, D. Amodei, and P. F. Christiano. Learning to summarize with human feedback. In *Advances in Neural Information Processing Systems*, 2020. <https://arxiv.org/abs/2009.01325>.
- [5] L. Ouyang et al. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems*, 2022. <https://arxiv.org/abs/2203.02155>.
- [6] S. Huang, M. Noukhovitch, A. Hosseini, K. Rasul, W. Wang, and L. Tunstall. The N+ Implementation Details of RLHF with PPO: A Case Study on TL;DR Summarization. arXiv:2403.17031, 2024. <https://arxiv.org/abs/2403.17031>.
- [7] Z. Wang, J. Zeng, O. Delalleau, H.-C. Shin, F. Soares, A. Bukharin, E. Evans, Y. Dong, and O. Kuchaiev. HelpSteer3-Preference: Open Human-Annotated Preference Data across Diverse Tasks and Languages. arXiv:2505.11475, 2025. <https://arxiv.org/abs/2505.11475>.
- [8] Qwen Team. Qwen2.5 Technical Report. arXiv:2412.15115, 2024. <https://arxiv.org/abs/2412.15115>.
- [9] E. J. Hu et al. LoRA: Low-Rank Adaptation of Large Language Models. In *International Conference on Learning Representations*, 2022. <https://arxiv.org/abs/2106.09685>.
- [10] L. von Werra et al. TRL: Transformer Reinforcement Learning. Hugging Face, 2020–2026. <https://github.com/huggingface/trl>; https://huggingface.co/docs/trl/ppo_trainer.
- [11] L. Gao, J. Schulman, and J. Hilton. Scaling Laws for Reward Model Overoptimization. In *Proceedings of the 40th International Conference on Machine Learning*, 2023. <https://arxiv.org/abs/2210.10760>.

-
- [12] R. Rafailov, A. Sharma, E. Mitchell, C. D. Manning, S. Ermon, and C. Finn. Direct Preference Optimization: Your Language Model is Secretly a Reward Model. In *Advances in Neural Information Processing Systems*, 2023. <https://arxiv.org/abs/2305.18290>.